

Mobile Number Validation using Exception Handling

Java Programming Laboratory Record — Built-in and User-defined Exceptions

1. Objective / Aim

To write a Java program that reads a mobile number from the user and validates it using **exception handling**. A valid mobile number must contain exactly **10 digits (0-9)**. The program must raise an `ArrayIndexOutOfBoundsException` if the length exceeds 10, a **user-defined** `LengthNotSufficientException` if the length is less than 10, and a `NumberFormatException` if any character other than a digit is present.

2. System Requirements

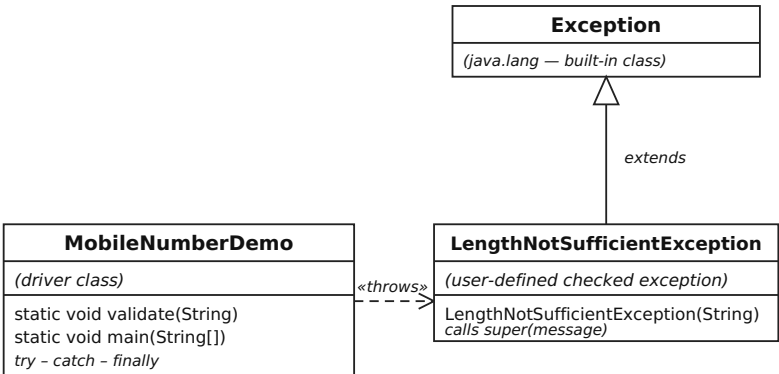
Hardware	Standard PC setup — Intel Core i3 / i5 processor or higher, 4 GB / 8 GB RAM, 256 GB storage, standard keyboard and monitor.
Operating System	Windows 10 / 11, Linux (Ubuntu) or macOS.
Software	Java Development Kit (JDK 17 or JDK 21).
Editor / IDE	Eclipse, IntelliJ IDEA, NetBeans or VS Code (any plain text editor with a terminal is sufficient).

3. Algorithm / Logic Description

- Start.**
- Create a user-defined checked exception class `LengthNotSufficientException` that extends `Exception` and passes the error message to the super class constructor using `super(message)`.
- In the public class `MobileNumberDemo`, define a static method `validate(String number)` that declares throws `LengthNotSufficientException`.
- Inside `validate()`, scan every character of the string using `charAt(i)` and `Character.isDigit()`. If any character is not a digit, **throw** a `NumberFormatException`.
- If the length of the string is greater than 10, throw an `ArrayIndexOutOfBoundsException`.
- If the length of the string is less than 10, throw the user-defined `LengthNotSufficientException`.
- If none of the above conditions is satisfied, the number is valid — display *Valid number*.
- In `main()`, read the mobile number using a `Scanner` object and call `validate()` inside a try block.
- Handle the three exceptions in three separate catch blocks and display the corresponding message using `getMessage()`.
- Close the `Scanner` object inside the finally block, since it is executed whether or not an exception occurs.
- Stop.**

Note on the order of checks: the digit check must be performed before the length checks. For an input such as `98ab@123` (length 8), testing the length first would wrongly report `LengthNotSufficientException` instead of the expected `NumberFormatException`.

4. UML Class Diagram



The hollow triangular arrow head denotes the extends relationship (generalisation). The dashed arrow denotes a dependency — the driver class throws and handles the user-defined exception. The built-in exceptions `NumberFormatException` and `ArrayIndexOutOfBoundsException` are unchecked and hence need not be declared.

5. Source Code

File name: **MobileNumberDemo.java** (saved inside the package folder **Excercise**)

```
package Excercise;
```

```
import java.util.Scanner;

// User-defined checked exception
class LengthNotSufficientException extends Exception {
    LengthNotSufficientException(String message) {
        super(message); // passes message to Exception class
    }
}

public class MobileNumberDemo {

    // Validates the number and throws the appropriate exception
    static void validate(String number)
        throws LengthNotSufficientException {

        // Rule 1: every character must be a digit (0-9)
        for (int i = 0; i < number.length(); i++) {
            if (!Character.isDigit(number.charAt(i))) {
                throw new NumberFormatException(
                    "Invalid Mobile Number - NumberFormatException");
            }
        }

        // Rule 2: length greater than 10
        if (number.length() > 10) {
            throw new ArrayIndexOutOfBoundsException(
                "Invalid Mobile Number - ArrayIndexOutOfBoundsException");
        }

        // Rule 3: length less than 10
        if (number.length() < 10) {
            throw new LengthNotSufficientException(
                "Invalid Mobile Number - LengthNotSufficientException");
        }

        System.out.println("Valid number");
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Mobile Number: ");
        String number = sc.nextLine().trim();

        try {
            validate(number);
        }
        catch (NumberFormatException e) { // non-digit character
            System.out.println(e.getMessage());
        }
        catch (ArrayIndexOutOfBoundsException e) { // more than 10 digits
            System.out.println(e.getMessage());
        }
        catch (LengthNotSufficientException e) { // fewer than 10 digits
            System.out.println(e.getMessage());
        }
        finally {
            sc.close(); // always executed
        }
    }
}
```

6. Compilation & Execution Commands

Save the file with the same name as the public class, i.e. **MobileNumberDemo.java**, inside a folder named **Excercise**. Open the terminal in the *parent* folder of Excercise and type:

```
// Compile the source file
javac Excercise/MobileNumberDemo.java

// Run the compiled bytecode using the fully qualified class name
java Excercise.MobileNumberDemo
```

If the package `Excercise`; statement is removed, the simpler commands `javac MobileNumberDemo.java` and `java MobileNumberDemo` may be used instead.

7. Output Screen

Case 1 — Valid mobile number (exactly 10 digits)

```
C:\Java> java Excercise.MobileNumberDemo
Enter Mobile Number: 9885089465
Valid number
```

Case 2 — Length greater than 10

```
C:\Java> java Excercise.MobileNumberDemo
Enter Mobile Number: 98567890121
Invalid Mobile Number - ArrayIndexOutOfBoundsException
```

Case 3 — Length less than 10

```
C:\Java> java Excercise.MobileNumberDemo
Enter Mobile Number: 88664433
Invalid Mobile Number - LengthNotSufficientException
```

Case 4 — Contains a character other than a digit

```
C:\Java> java Excercise.MobileNumberDemo
Enter Mobile Number: 98ab@123
Invalid Mobile Number - NumberFormatException
```

8. Result

The Java program to validate a mobile number using built-in and user-defined exceptions was written, compiled and executed successfully, and the output was verified for all four cases — valid number, excess length, insufficient length and invalid character.